HANSUNG UNIVERSITY

AI가 코드를 쓰는 시대, 왜 Python을 배워야 할까?

한성대학교 컴퓨터공학부
이청용 교수

Part 1. AI로 달라진 코딩 환경

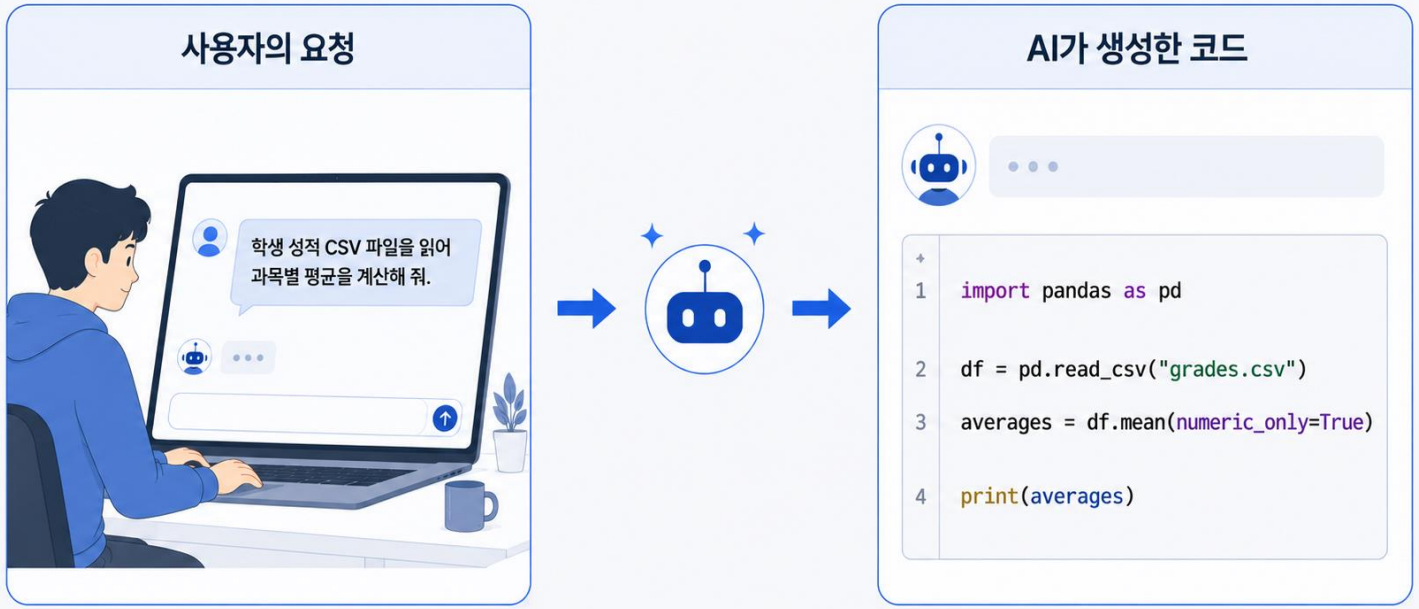
여러분은 코딩할 때 AI를 얼마나 사용하고 있습니까?

- **생성형 AI가 코딩 과정의 일상적인 보조 도구로 빠르게 확산**
  - 과제 전체 작성부터 오류 해결과 개념 참고까지 다양한 방식으로 활용
- **중요한 차이는 AI 사용 여부보다 사용 목적과 의존 수준에서 발생**



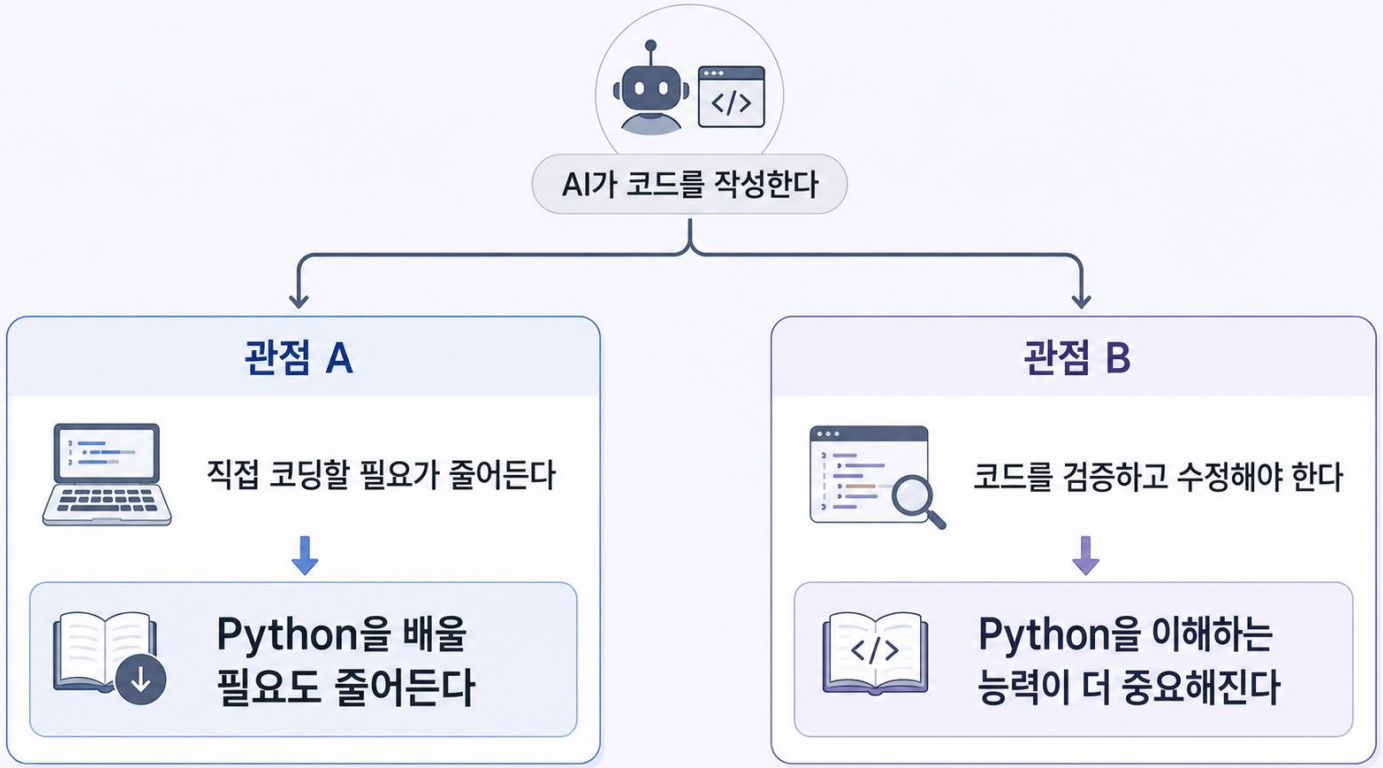
AI는 이미 몇 초 만에 Python 코드를 만든다

- **자연어로 원하는 작업을 설명하면 AI가 실행 가능한 Python 코드 생성**
  - 예: 파일 읽기, 데이터 분석, 결과 출력 등의 절차를 몇 초 안에 구현 가능
- **빠른 코드 생성이 결과의 정확성까지 보장하는 것은 아님**



AI가 코드를 잘 쓸수록 Python은 덜 중요해질까?

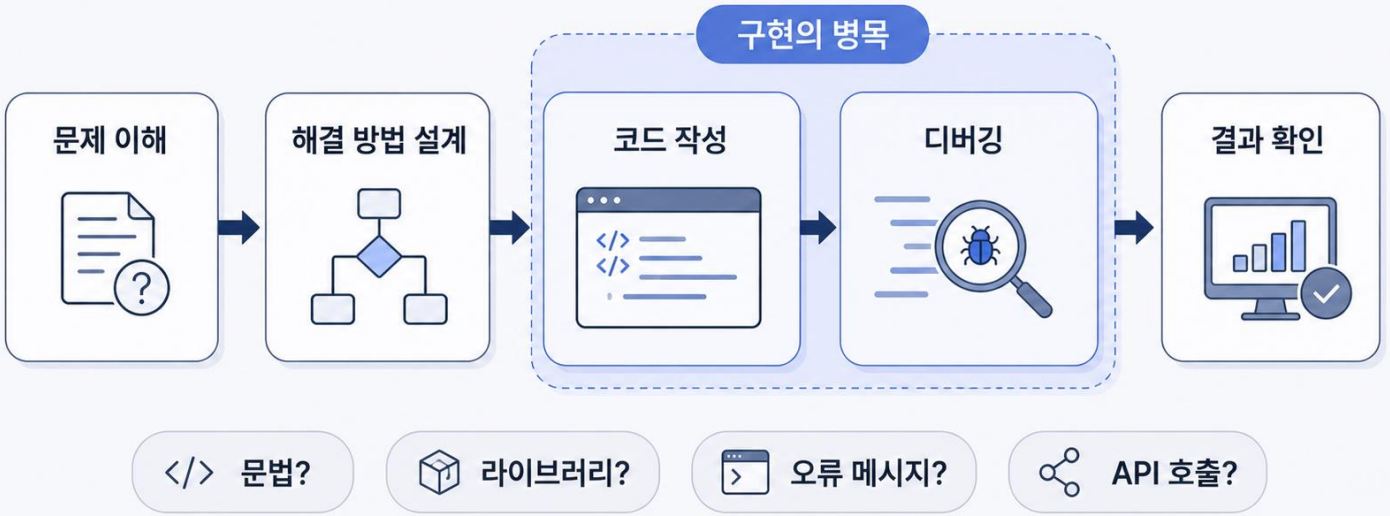
- **AI 활용으로 코드를 직접 입력해야 하는 부담은 감소**
  - 반복 문법과 기본 코드 작성에 투입되는 시간 절약
- **생성된 코드를 이해하고 검증하는 Python 역량의 중요성은 증가**
  - 잘못된 조건과 계산 방식을 발견하려면 코드의 의미 파악 필요



Part 2. 코드 작성에서 문제 해결로

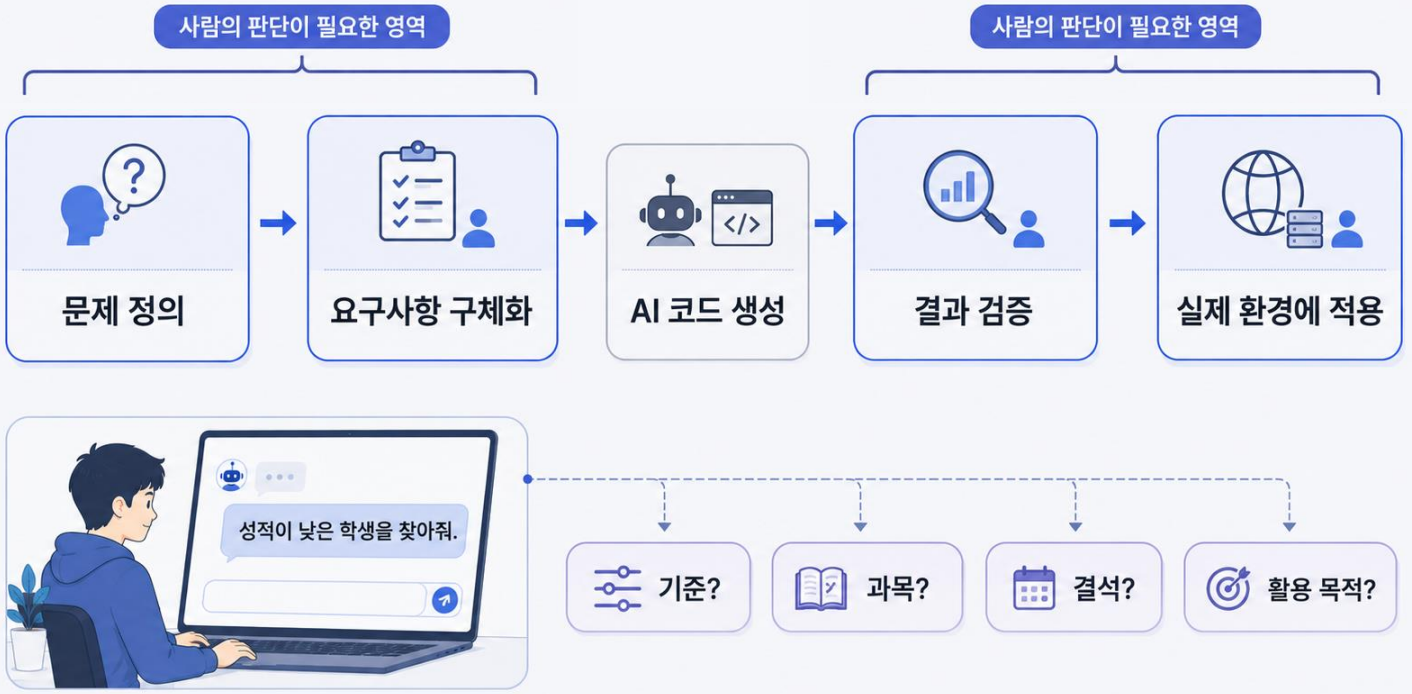
과거의 병목은 아이디어를 코드로 구현하는 것이었다

- **과거에는 해결 방법을 실제 코드로 옮기는 과정이 주요 병목**
  - 문법, 라이브러리, 오류 메시지, API 사용법을 직접 파악해야 구현 가능
- **코드 작성과 디버깅에 많은 시간이 필요해 아이디어 실험 속도 제한**



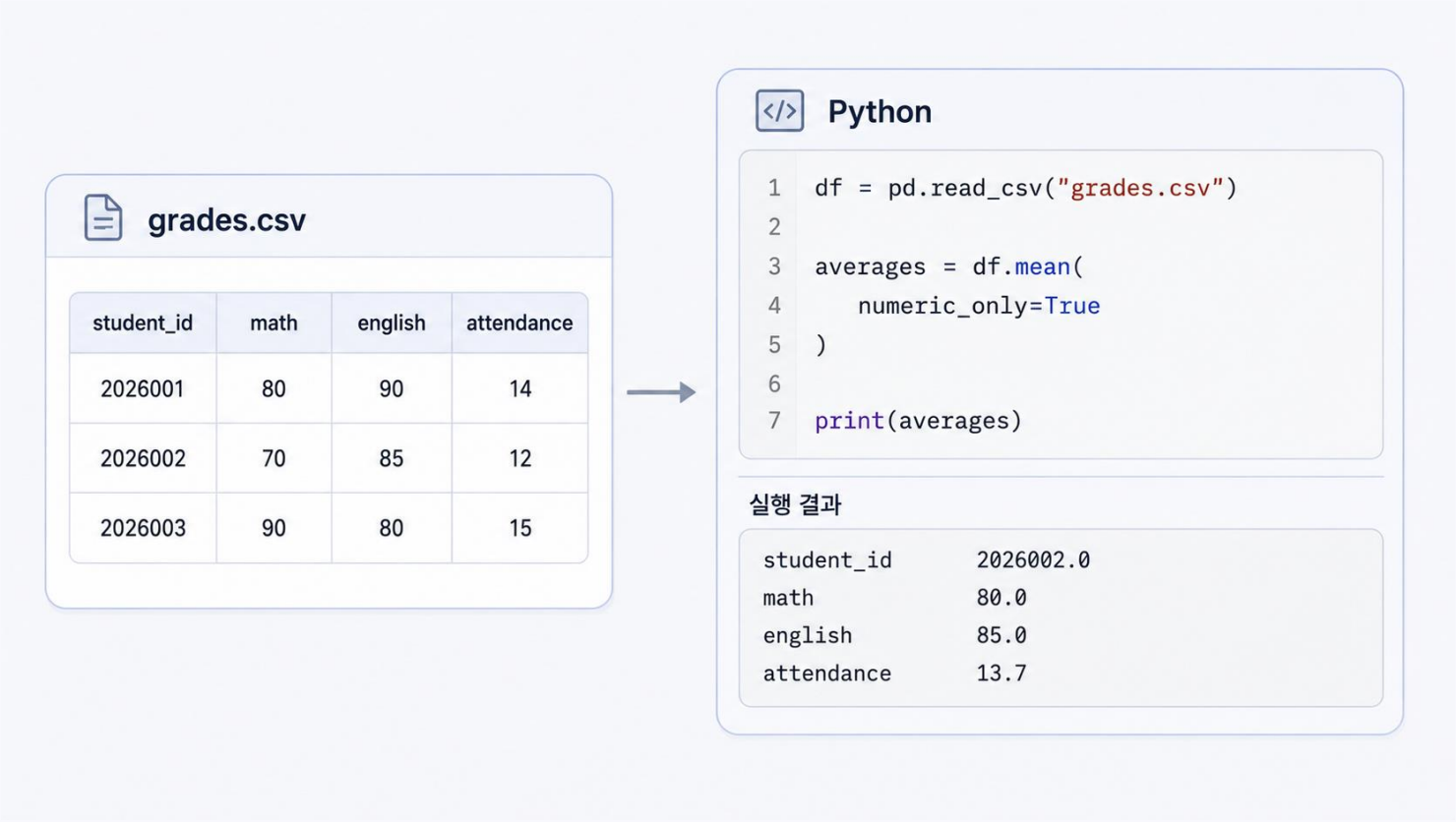
이제 병목은 '어떻게 만들까'에서 '무엇을 만들까'로 이동한다

- **AI가 코드 생성을 지원하면서 사람의 핵심 역할이 문제 정의로 이동**
  - 예: 분석 기준, 대상 과목, 결측값 처리, 활용 목적 등을 사람이 구체화
- **생성된 결과가 요구사항에 맞는지 검증하는 판단은 여전히 사람의 책임**
  - 실제 환경 적용 가능성과 결과의 의미까지 확인 필요



실행에 성공한 코드가 문제 해결에 성공한 것은 아니다

- **오류 없이 실행된 코드는 문법적으로 동작했다는 의미에 불과**
  - 예: 학생번호나 출석 횟수의 평균처럼 계산 가능하지만 의미 없는 결과 발생 가능
- **문제 해결의 성공 여부는 계산 대상과 결과의 타당성으로 판단**
  - 코드 실행 결과와 원래 분석 목적의 일치 여부 확인 필요



컴퓨터공학자의 역할은 코드 작성자를 넘어선다

- **컴퓨터공학자의 역할은 코드 작성에서 문제 해결 과정 전체의 설계로 확장**
  - 문제 정의, 작업 분해, 검증, 시스템 연결을 종합적으로 수행
- **AI는 코드 생성에 강하지만 최종 방향과 품질 판단은 사람이 담당**
  - 사람과 AI의 역할을 구분하고 적절하게 연결하는 역량 필요



가장 좋은 언어가 아니라 목적에 맞는 언어를 선택한다

- **프로그래밍 언어는 장단점보다 해결하려는 문제에 따라 선택**
  - 하드웨어와 성능 제어에는 C/C++, 대규모 서비스에는 Java·Kotlin 활용
- **Python은 데이터 분석, AI 실험, 자동화와 빠른 시제품 제작에 적합**
  - 간결한 문법과 풍부한 라이브러리를 통해 아이디어의 빠른 실행 가능



Part 3. Python이 필요한 이유

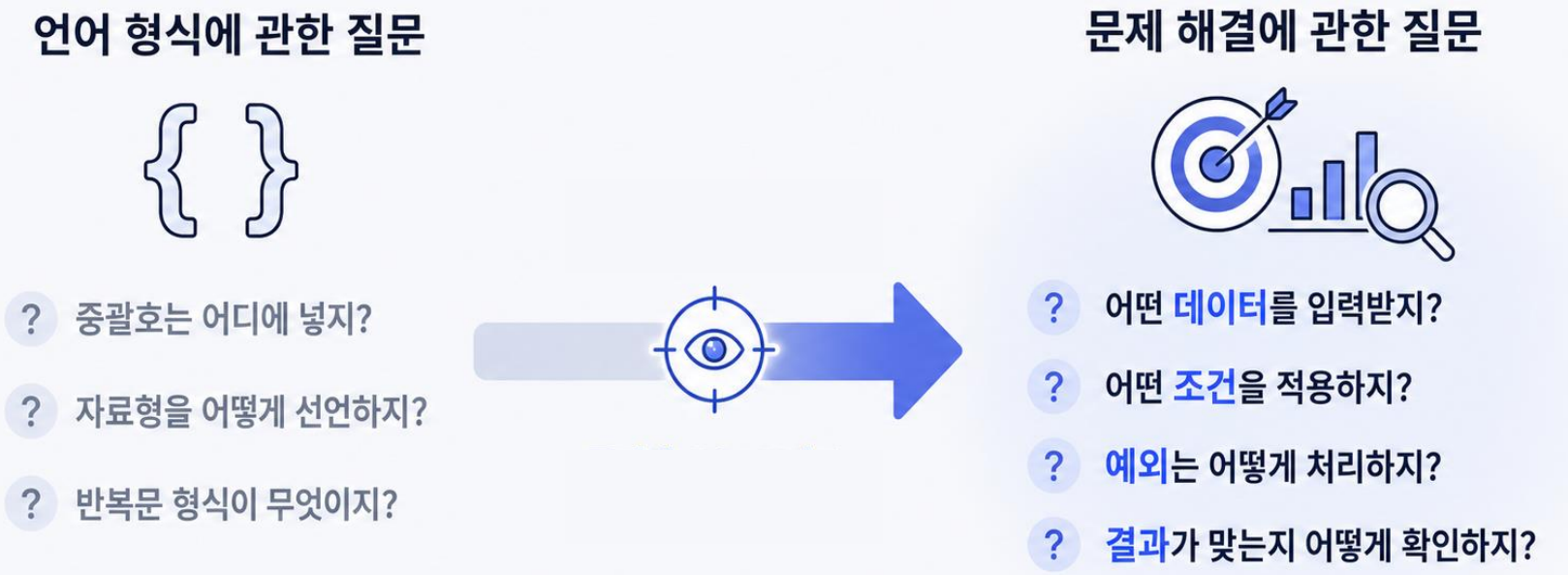
Python은 아이디어와 실행 사이의 거리를 줄인다

- **Python은 자연어로 표현한 문제를 짧은 절차와 코드로 연결**
  - 예: 조건 확인, 점수 비교, 대상 선택의 논리를 간결하게 표현 가능
- **코드 작성 부담이 낮아지면서 해결 절차와 조건 설계에 집중 가능**



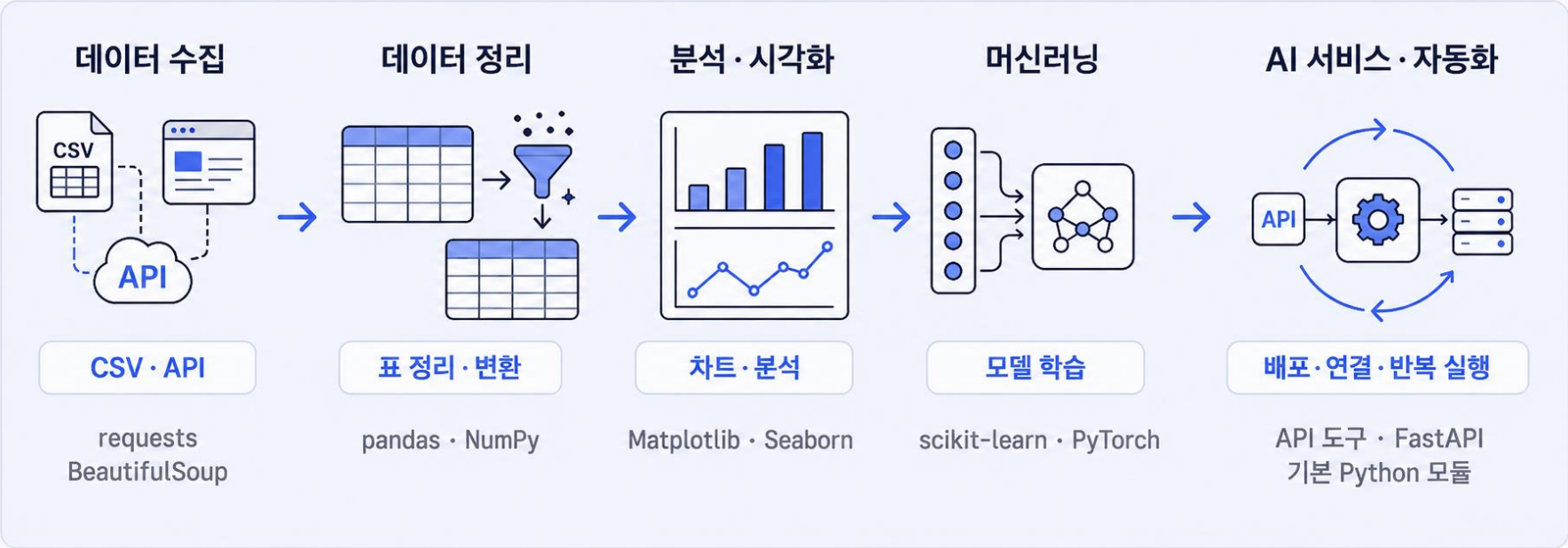
문법 부담이 낮아지면 문제 자체에 더 집중할 수 있다

- **문법적 부담이 줄어들면 코드 구현보다 문제 정의와 해결에 더 집중 가능**
  - 예: 어떤 데이터를 사용할지, 어떤 조건을 적용할지, 예외를 어떻게 처리할지 검토
- **결과가 실제 질문에 올바르게 답하는지 확인하는 사고 역량 강화**



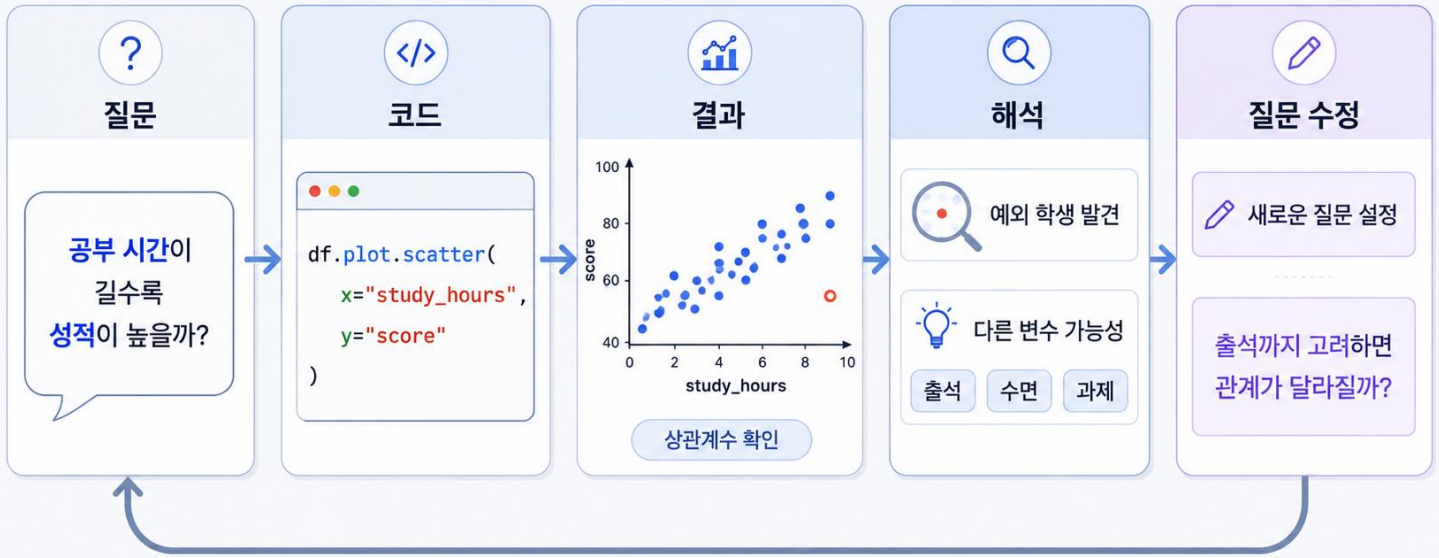
Python 하나로 데이터에서 AI까지 이어갈 수 있다

- **Python 생태계 안에서 데이터 수집부터 AI 서비스까지 하나의 흐름으로 연결**
  - 웹과 API 수집, 데이터 정리, 분석·시각화, 머신러닝 수행 가능
- **동일한 언어로 분석 결과를 서비스와 자동화 기능으로 확장 가능**
  - FastAPI 등의 도구를 이용한 배포와 반복 실행 지원



Python은 생각을 바로 시험해 보는 도구다

- **Python은 질문을 코드로 바꾸고 결과를 즉시 확인하는 실험 도구**
  - 예: 공부 시간과 성적의 관계처럼 가설을 데이터로 빠르게 확인 가능
- **결과 해석을 통해 새로운 질문을 만들고 다시 코드를 수정하는 반복 학습 가능**



Python의 강점을 이해하려면 한계도 알아야 한다

- **Python은 데이터 분석과 AI 실험, 자동화, 빠른 시제품 제작에 강점**
  - 여러 시스템을 연결하는 풍부한 라이브러리와 높은 생산성 제공
- **높은 실행 성능이나 세밀한 하드웨어 제어가 필요한 분야에는 한계 존재**
  - 목적에 따라 C/C++, Java, JavaScript 등 다른 언어와의 조합 필요



Python은 AI와 함께 문제를 해결하기에 적합한 언어다

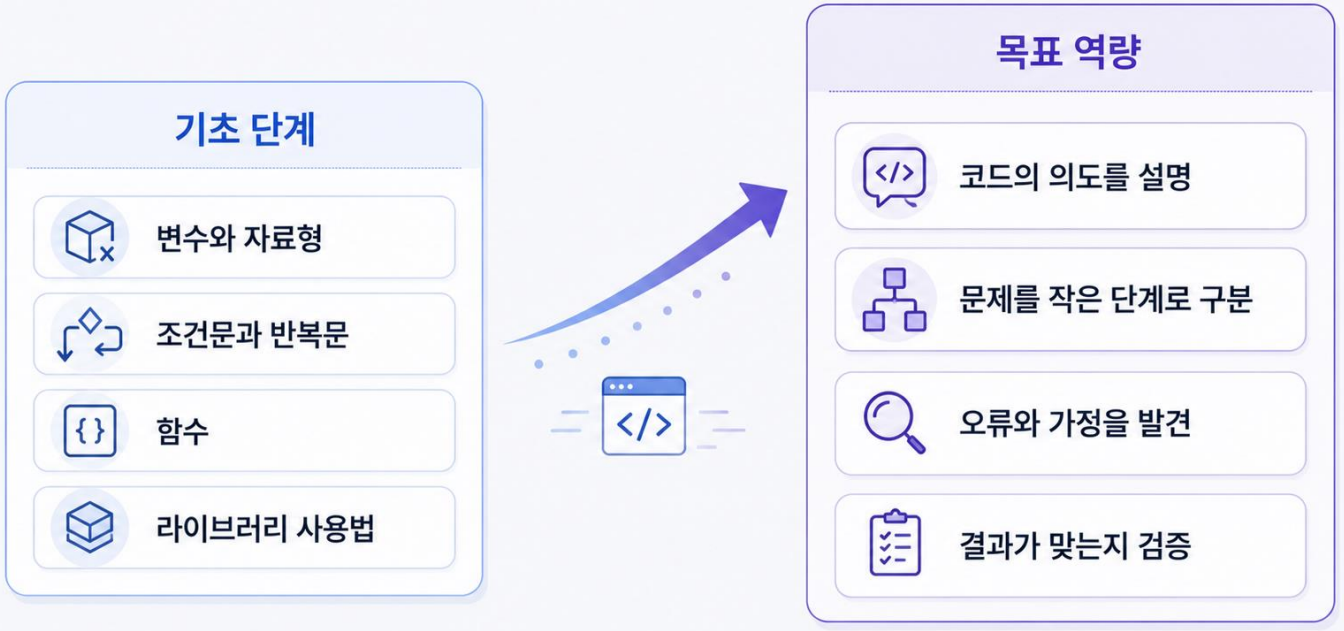
- **사람은 문제 정의와 코드 검토, 결과 검증, 도구 연결을 담당**
  - 무엇을 만들고 어떤 기준으로 판단할지 결정하는 역할 수행
- **Python은 간결한 표현과 높은 가독성으로 검토와 수정 지원**
  - 데이터·AI 생태계와 빠른 실행 환경을 통해 사람과 AI의 협업 촉진



Part 4. 문법을 넘어선 문제 해결 역량

문법은 출발점이지만 학습의 최종 목표는 아니다

- **변수, 조건문, 반복문, 함수와 라이브러리는 문제 해결을 위한 기초 수단**
  - 문법 자체의 암기보다 필요한 코드를 읽고 활용하는 능력이 중요
- **최종 학습 목표는 코드의 의도 설명과 오류·가정의 발견**
  - 문제를 작은 단계로 나누고 결과의 정확성을 검증하는 역량 확보



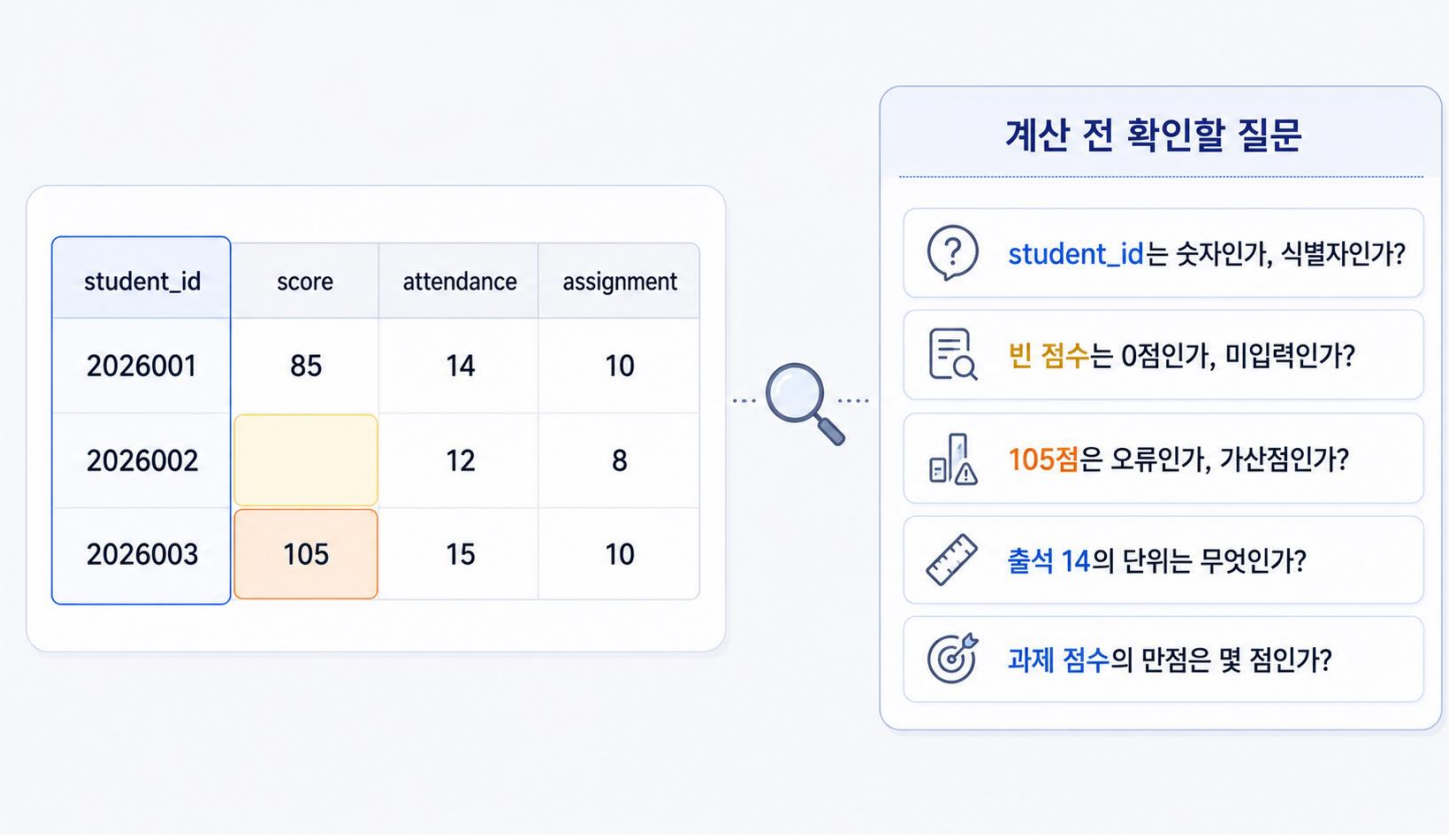
좋은 코드는 좋은 문제 정의에서 시작한다

- **좋은 코드는 입력·처리·출력의 기준을 명확히 정의한 문제에서 출발**
  - 입력 데이터와 분석 조건이 구체적일수록 코드의 목적도 명확해짐
- **예외와 데이터 기준을 먼저 결정해야 신뢰할 수 있는 결과 생성**
  - 결측값, 허용 범위, 대상 선정 조건을 처리 과정에 반영



숫자를 계산하기 전에 데이터의 의미를 읽어야 한다

- **계산 전에 각 열이 나타내는 의미와 값의 범위를 먼저 확인**
  - 예: 학생번호는 식별자인지 숫자인지, 빈 점수는 0점인지 결측값인지 구분 필요
- **이상값과 분석 기준은 자동 계산만으로 결정 불가**
  - 예: 105점의 처리 방식, 출석 단위, 과제 점수의 만점 등을 맥락에 따라 판단



AI가 만든 코드는 실행보다 검증이 먼저다

- **AI가 만든 코드는 정상값뿐 아니라 경계값과 예외값까지 검증 필요**
  - 예: 기준점 60점과 바로 아래인 59점에서 조건이 의도대로 작동하는지 확인
- **결측값과 허용 범위 밖의 데이터에는 별도의 처리 기준 필요**
  - 예: None이나 105점 입력을 그대로 판정하지 않고 오류 또는 확인 대상으로 분류

is_pass()

```
def is_pass(score):
    return score >= 60
```



테스트 사례

	입력	기대 결과	확인할 내용
정상값	75	True	일반적인 값
경계값	60	True	경계값
	59	False	경계 바로 아래
예외·범위 확인	None	오류 처리	결측값
	105	정책 확인	범위 밖의 값

가치는 한 줄 코드가 아니라 끝까지 동작하는 흐름에서 나온다

- **실제 가치는 데이터 입력부터 최종 활용까지 이어지는 전체 흐름에서 생성**
  - 수집·정리·분석·시각화·보고서 생성·의사결정이 하나의 과정으로 연결
- **AI는 코드 생성과 수정, 설명, 테스트를 지원하지만 판단 책임은 사람이 담당**
  - 분석 목적, 데이터 의미, 결과 검증과 최종 사용 기준을 사람이 결정



Part 5. AI 코드의 작성과 검증

한 문장 요청에는 결정되지 않은 것이 많다

- **한 문장으로 작성된 요청에는 코드에 필요한 세부 조건이 충분히 포함되지 않음**
  - 예: 평균 대상 과목, 지원 기준, 결측값 처리, 그래프 종류 등이 미결정 상태
- **코드를 요청하기 전에 데이터와 규칙, 예외, 출력 형식을 구체화할 필요**
  - 결정되지 않은 조건이 많을수록 AI의 임의 가정도 증가



AI는 비어 있는 요구사항을 가정으로 채운다

- **AI는 명시되지 않은 요구사항을 임의의 코드 조건으로 보완**
  - 예: 숫자형 열 전체를 평균에 포함하거나 평균 70점 미만을 지원 기준으로 가정
- **코드가 실행되더라도 이러한 가정이 실제 문제의 의미와 다를 가능성 존재**
  - 예: 학생번호 포함과 같은 부적절한 계산이나 근거 없는 대상 선정 발생 가능

AI가 생성한 코드

```python
import pandas as pd

df = pd.read_csv("grades.csv")

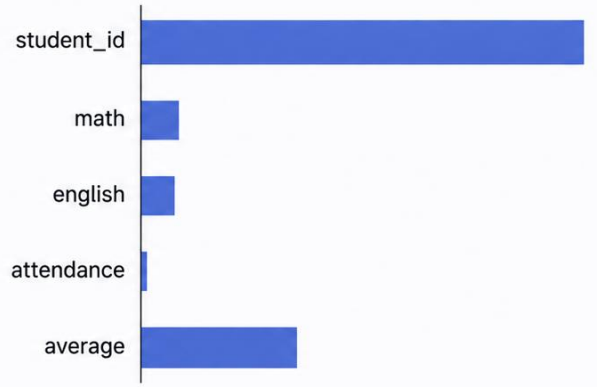
df["average"] = df.mean(
    axis=1,
    numeric_only=True
)

support = df[df["average"] < 70]

df.mean(numeric_only=True).plot.bar()
```

실행 결과

지원 필요 학생: 0명



Python을 알아야 AI의 가정을 구체적으로 지적할 수 있다

- **Python을 이해하면 코드 표현과 실제 동작의 차이를 구체적으로 분석 가능**
  - 예: numeric_only=True가 모든 숫자 열을 계산 대상으로 선택한다는 점 확인
- **코드가 동작하는 방식과 문제의 의미가 일치하는지 별도로 검증**
  - 평균 70점 미만이라는 조건의 근거와 과목 선택 방식의 타당성 점검

```python
df["average"] = df.mean(
    axis=1,
    numeric_only=True
)

support = df[df["average"] < 70]
```



좋은 프롬프트는 말재주가 아니라 실행 명세다

- **좋은 프롬프트는 데이터, 규칙, 예외, 출력 조건을 포함한 실행 명세**
  - 사용할 열과 계산 방식, 대상 선정 기준을 명확하게 제시
- **결측값 처리와 표시 정보까지 지정해야 해석 가능한 결과 생성**
  - 예: 학번, 평균, 선정 이유를 제공하고 개인정보가 불필요하게 노출되지 않도록 제한



명시적인 기준은 검토 가능한 코드를 만든다

- **명시적인 요구사항은 각각 확인 가능한 변수와 조건문으로 변환**
  - 예: 과목 목록, 결측값 처리, 평균 기준, 출석 기준을 코드에 직접 표현
- **조건이 코드에 드러나면 AI의 가정과 구현 오류를 쉽게 추적 가능**
  - 요구사항과 코드의 대응 관계를 항목별로 검토 가능

</> 개선된 Python 코드

```python
subjects = ["math", "english"]
missing = df[subjects].isna().any(axis=1)

df["average"] = df[subjects].mean(axis=1)

support = (
    ~missing
    & (
        (df["average"] < 70)
        | (df["attendance"] < 12)
    )
)
```

명세		코드
과목은 math, english	→	`subjects = ["math", "english"]`
결측값은 분리	→	`missing = ...`
평균 70점 미만	→	`average < 70`
출석 12회 미만	→	`attendance < 12`

신뢰는 좋은 프롬프트가 아니라 테스트에서 만들어진다

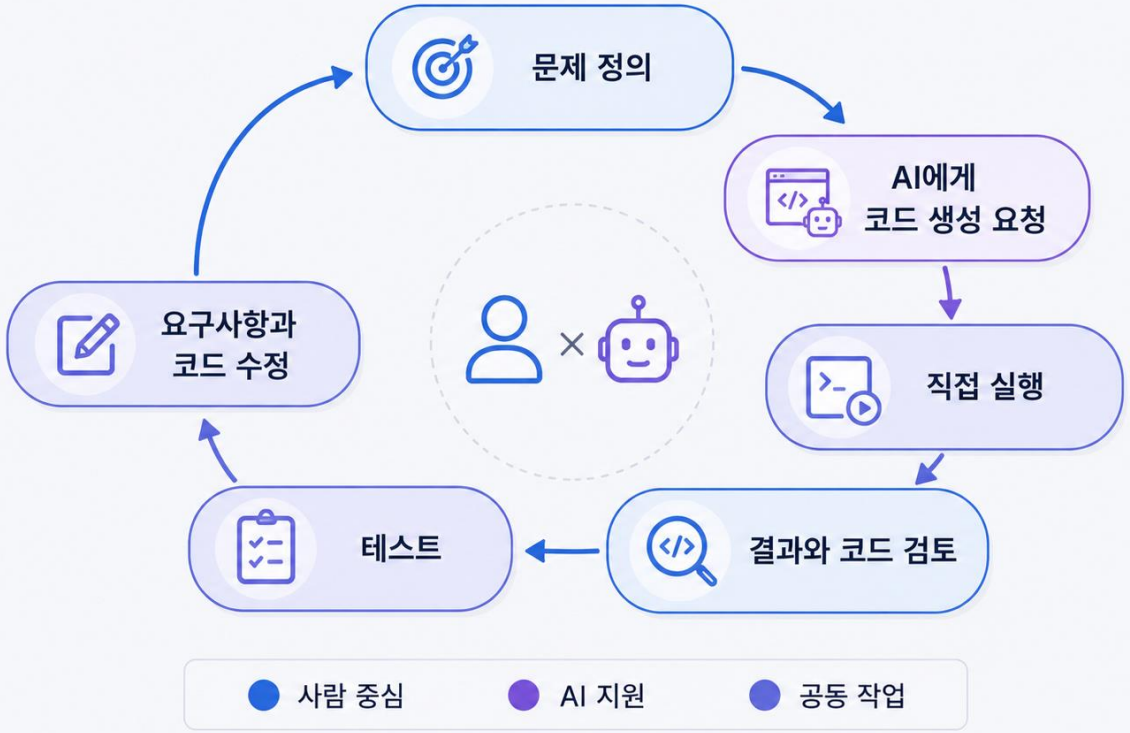
- **테스트는 정상값과 경계값에서 코드가 요구사항대로 작동하는지 확인**
  - 기준을 충족하는 사례와 정확히 경계에 있는 사례를 함께 검증
- **출석 부족, 결측값, 허용 범위 밖 데이터까지 포함한 예외 테스트 필요**
  - 오류를 숨기지 않고 확인 요청이나 정책 확인 대상으로 처리

테스트 사례

사례	입력 상황	기대 결과
정상값	평균 65, 출석 14	지원 대상
경계값	평균 70, 출석 14	지원 대상 아님
출석 부족	평균 85, 출석 11	지원 대상
결측값	영어 점수 없음	확인 필요
데이터 오류	수학 점수 120	오류 또는 정책 확인

좋은 AI 코딩은 요청→실행→검증→수정의 반복이다

- **좋은 AI 코딩은 문제 정의부터 수정까지 반복되는 협업 과정**
  - 사람이 문제를 정의하고 AI에 코드 생성을 요청한 뒤 직접 실행
- **실행 결과와 코드를 검토하고 테스트를 통해 문제점 발견**
  - 발견된 오류와 누락 조건을 요구사항과 코드에 다시 반영



Part 6. 학습에서 프로젝트로

Python은 읽기→검증→프로젝트의 순서로 성장한다

- **첫 단계는 AI가 생성한 코드를 읽고 필요한 부분을 수정하는 역량 확보**
  - 변수, 자료형, 조건문, 함수와 오류 메시지의 의미 이해
- **다음 단계는 데이터 처리 결과를 검증하고 시각화하는 경험 축적**
  - 파일 입출력, pandas, 결측값·이상값 처리와 테스트 수행
- **최종 단계는 여러 기능을 연결한 작은 시스템의 완성**
  - 외부 API, 업무 자동화, 사용자 입력과 결과 출력을 하나의 프로젝트로 통합



작은 프로젝트에서 Python 지식이 하나로 연결된다

- **생활 속 질문을 공공데이터로 분석하며 Python 지식을 실제 문제 해결에 적용**
  - 예: 대중교통, 생활인구, 공공자전거, 청년 인구 등 관심 주제를 분석 가능한 질문으로 구체화
- **데이터 수집부터 결과 발표까지 수행하며 Python 지식을 하나의 프로젝트로 연결**
  - 데이터 정리, 분석·시각화와 테스트를 거쳐 재실행 가능한 결과 완성
  - 분석 근거와 그래프, 결론을 정리해 3분 발표로 공유



AI가 코드를 쓸수록, 코드를 이해하는 사람이 방향을 결정한다

- **Python 지식은 문제를 계산 가능한 조건과 절차로 변환하는 기반**
  - 모호한 요청을 데이터, 규칙, 예외와 출력 형태로 구체화
- **생성된 코드와 결과를 검증해 AI의 오류와 숨은 가정을 발견**
  - 실행 성공 여부를 넘어 계산의 의미와 기준의 타당성 확인
- **데이터와 도구를 연결해 실제 문제를 해결하는 전체 흐름 완성**



HSU HANSUNG UNIVERSITY

수고하셨습니다.